Branching

Topic: If-Else Statements

Key Vocabulary

- Equality operator `==`
  - Returns `True` when the left side and right side are equal.
- Inequality operator `!=`
  - Means “not equal.”
- Indentation
  - Statements inside a branch must be indented, usually by 4 spaces.

Basic Idea

- `if` executes a block when a condition is `True`
- `else` executes a different block when the condition is `False`

Example

```
name = "Armando"

if name == "Armando":
    print("1st execution")
else:
    print("2nd execution")
```

Multi-Branch If-Elif-Else

Definition

An `if-else` statement can be extended to include 3 or more branches using:

- `if`
- `elif`
- `else`

`elif` means “else if.”

How it works

- Python checks each condition in order.
- As soon as one condition is `True`, that branch runs.
- If none are `True`, the `else` branch runs.

Example

```
if expression1:
    statements_for_first_branch
elif expression2:
    statements_for_second_branch
else:
    statements_for_else_branch
```

Detecting Ranges with Branches

Relational Operators

Definition

A relational operator compares two values and checks how they relate.

Examples:

- `<`
- `>`
- `==`

Table of Relational Operators

Operator	Meaning	Example (`x = 3`)
`<`	less than	`x < 4` → `True`, `x < 3` → `False`
`>`	greater than	`x > 2` → `True`, `x > 3` → `False`
`<=`	less than or equal to	`x <= 4` → `True`, `x <= 3` → `True`, `x <= 2` → `False`
`>=`	greater than or equal to	`x >= 2` → `True`, `x >= 3` → `True`, `x >= 4` → `False`

Operator Chaining

Python supports chaining comparison operators.

Example

```
a < b < c
```

How it works

- Comparisons are checked from left to right.
- First, Python checks `a < b`
- If that is `True`, Python checks `b < c`
- If the first comparison is `False`, Python stops
- `a` is not directly compared to `c`

Logical Operators

Definition

Logical operators treat expressions as `True` or `False`.

Main Logical Operators

- `and`
- `or`
- `not`

Boolean Values

A Boolean is a value that is either:

- `True`
- `False`

These are Python keywords, so they must be capitalized.

Table of Logical Operators

Operator	Meaning
`a and b`	`True` only if both operands are `True`
`a or b`	`True` if at least one operand is `True`
`not a`	Reverses the Boolean value

Example

```
if statement1:
    print("....")
elif statement2 and statement3:
    print("....")
else:
    print("....")
```

Comparing Data Types and Common Errors

Numbers

- Numbers are compared mathematically.

Strings

- Strings are compared character by character.
- Each character is compared using its numeric value (ASCII or Unicode).
- Common operators:
  - `==`
  - `!=`

Lists and Tuples

- Lists and tuples are compared in order, element by element.
- The result is decided by the first mismatched element.

Dictionaries

- Dictionaries can only be compared with:
  - `==`
  - `!=`
- Two dictionaries are equal only if:
  - they have the same keys
  - each key has the same corresponding value

Membership and Identity Operators

Membership Operators

Definition

Membership operators check whether a value exists inside a container.

Operators:

- `in`

- `not in`

They return `True` or `False`.

Example: `in`

```
names_in_class = ["Armando", "Mace", "Gian"]
name = input("Input name: ")

if name in names_in_class:
    print(f"Found {name} on roster.")
else:
    print("Not in roster.")
```

Example: `not in`

```
names_in_class = ["Armando", "Mace", "Gian"]
name = input("Input name: ")

if name not in names_in_class:
    print(f"{name} not in roster.")
else:
    print(f"Found {name} in roster.")
```

Substrings with `in` / `not in`

You can also check whether part of a string appears inside a larger string.

Examples

```
if "/1.1" in request_str:
    print("HTTP protocol 1.1")
```

```
if "HTTPS" not in request_str:
    print("Unsecured connection")
```

Membership in Dictionaries

For dictionaries, membership checks keys, not values.

Example

```
my_dict = {"A": 1, "B": 2, "C": 3}

if "B" in my_dict:
    print('Found "B"')
else:
    print('"B" not found')
```

Identity Operators: `is` and `is not`

Definition

- `is` checks whether two variables refer to the exact same object in memory
- `is not` checks whether they refer to different objects

Important

- `==` compares values
- `is` compares identity

Example idea

```
x is y
```

This is `True` only if `x` and `y` point to the same object.

`id()` Function

You can use `id()` to see an object's identifier.

```
id(x)
id(y)
```

If:

```
x is y
```

then:

```
id(x) == id(y)
```

Order of Evaluation

Python evaluates expressions in this order:

1. `()`
   - Parentheses first
2. , `/`, `%`, `+`,
   - Arithmetic operators
3. `<`, `<=`, `>`, `>=`, `==`, `!=`
   - Relational and equality operators
4. `not`
   - Logical NOT
5. `and`
   - Logical AND
6. `or`
   - Logical OR

Conditional Expressions

A conditional expression is a shorter way to write a simple `if-else`.

Standard Form

```
if condition:
    my_var = expr1
else:
    my_var = expr2
```

Conditional Expression Form

```
my_var = expr1 if condition else expr2
```

Quick Summary

Branching

- `if` runs code when a condition is `True`
- `else` runs code when it is `False`
- `elif` adds more conditions

Comparison Tools

- Relational operators compare values
- Logical operators combine conditions
- Membership operators check whether something exists in a container
- Identity operators check whether two variables point to the same object

Useful Reminder

- `==` checks value
- `is` checks memory identity